ARQUITETURA E DESENHO DE SOFTWARE

Battle Class

Requisitos Funcionais e Não Funcionais

Grupo 6

Universidade de Brasília — 2026/1

Sumário

1 Stakeholders identificados **1**

2 Requisitos funcionais elicitados **1**

2.1 Autenticação e acesso 1

2.2 Navegação principal 2

2.3 Modo estudo / questões 2

2.4 Pontuação e moedas de questão 3

2.5 Tower Defense 3

2.6 Moeda in-game e economia 4

2.7 Heróis 5

2.8 Perfil, estatísticas e ranking 5

2.9 Banco de questões e administração 5

3 Regras de negócio elicitadas **6**

4 Requisitos não funcionais elicitados **7**

5 Lacunas e ambiguidades **8**

1 Stakeholders identificados

Primários

- **Estudante/jogador:** usuário principal da plataforma.

Secundários

- **Admin/Curador:** gerencia o banco de questões e a qualidade do conteúdo.
- **IA:** gera questões para alimentar o banco.
- **Professor:** pode recomendar a plataforma aos alunos.

2 Requisitos funcionais elicitados

2.1 Autenticação e acesso

`RF01`
O sistema deve permitir cadastro de usuário.

`RF02`
O sistema deve permitir login com e-mail e senha.

`RF03`
O sistema deve permitir login com Google.

`RF04`
O sistema deve permitir logout.

`RF05`
O cadastro deve coletar ao menos nome, e-mail, senha e confirmação de senha.

`RF06`
O cadastro deve exigir aceite de termos (checkbox).

`RF07`
A senha deve conter no mínimo 8 caracteres, incluindo ao menos 1 letra maiúscula, 1 minúscula e 1 símbolo.

`RF08`
O sistema deve permitir recuperação de senha via e-mail.

Base: fluxo, escopo do MVP e lista de telas.

2.2 Navegação principal

RF09
Após autenticação, o usuário deve acessar a Home/Menu principal.

RF10
A Home deve exibir avatar, saldo de moedas, estatísticas, acesso ao modo estudo e ao Tower Defense.

RF11
O sistema deve possuir navegação para Perfil, Carteira e Ranking.

2.3 Modo estudo / questões

RF12
O usuário deve poder escolher uma categoria de prova, como ENEM, PAS/UnB, concursos, vestibulares e certificações técnicas.

RF13
Após selecionar a prova, o sistema deve sortear uma matéria por meio de uma roleta. O sorteio é aleatório, porém matérias já respondidas na sessão têm menor chance de serem sorteadas.

RF14
O sistema deve exibir uma questão com 5 alternativas de resposta.

RF15
O sistema deve classificar cada questão com um nível de dificuldade: Fácil (0,0–0,2), Médio (0,2–0,5), Difícil (0,5–0,8) ou Muito Difícil (0,8–1,0), armazenado como valor decimal de 0 a 1 no banco de dados.

RF16
O sistema deve exibir cronômetro durante a resolução da questão.

RF17
O sistema deve exibir o valor da questão em moedas.

RF18
O sistema deve permitir resposta indefinida de questões, sem limite por sessão.

RF19
Ao acertar, o sistema deve informar o acerto e a quantidade de moedas ganhas.

RF20
Ao errar, o sistema deve exibir a resposta correta e a resolução/explicação.

RF21
Após o feedback, o usuário deve poder seguir para a próxima questão ou voltar ao menu.

RF22
O sistema deve manter um histórico de respostas do usuário, registrando questão, resposta escolhida, acerto/erro e data.

2.4 Pontuação e moedas de questão

RF23
O sistema deve calcular moedas de questão com valor decrescente ao longo do tempo.

RF24
O sistema deve aplicar limite de 5 minutos por questão, após o qual o valor atinge o mínimo.

RF25
O sistema deve atribuir maior valor de moedas para questões mais difíceis.

RF26
Cada moeda ganha em questões deve equivaler a um ponto no ranking.

RF27
As moedas de questão devem persistir na carteira do usuário entre sessões e partidas.

2.5 Tower Defense

RF28
O sistema deve permitir ao usuário iniciar o modo Tower Defense a partir do menu principal.

RF29
O usuário deve poder selecionar 1 entre 3 mapas com dificuldades distintas.

RF30
O modo Tower Defense deve ser PvE.

RF31
A partida deve conter ondas progressivas de inimigos.

RF32
O sistema deve disponibilizar torres comuns compradas com moeda in-game.

RF33
O sistema deve disponibilizar torres desbloqueáveis com moeda de questão.

RF34
O sistema deve disponibilizar upgrades em árvore para torres.

RF35
Ao final da partida, o usuário deve retornar ao menu.

2.6 Moeda in-game e economia

RF36
O sistema deve conceder ouro in-game ao jogador ao eliminar inimigos durante a partida.

RF37
O ouro in-game não deve persistir entre partidas.

RF38
O sistema deve permitir gastar moedas de questão para: evoluir heróis, desbloquear novas torres e comprar powerups para níveis do Tower Defense.

RF39
O custo de powerups deve aumentar progressivamente conforme o nível do Tower Defense avança.

RF40
O gasto de moedas de questão deve ser permanente, sem devolução.

2.7 Heróis

RF41
O sistema deve permitir evolução de heróis com moedas de questão.

RF42
O jogador deve escolher qual herói utilizar em cada nível do Tower Defense.

RF43
Os heróis devem poder ser posicionados no mapa no início da partida.

RF44
Heróis devem conceder poderes e bônus às torres ao redor.

2.8 Perfil, estatísticas e ranking

RF45
O sistema deve disponibilizar perfil do usuário.

RF46
O perfil deve apresentar estatísticas do jogador.

RF47
O sistema deve exibir ranking global.

RF48
O ranking deve ser baseado exclusivamente nas moedas obtidas em questões.

RF49
O sistema deve destacar a posição do usuário no ranking.

RF50
O sistema deve permitir que o usuário edite seu perfil.

2.9 Banco de questões e administração

Esses requisitos aparecem mais como necessidade de negócio do que como fluxo detalhado do MVP, então trato como requisitos inferidos:

RF51
O sistema deve possuir banco de questões organizado por taxonomia: área, linha, disciplina e tema.

RF52
O sistema deve permitir alimentação do banco de questões por administradores/curadores.

RF53
O sistema deve permitir uso de IA para geração e revisão de questões.

RF54
O conteúdo gerado por IA deve poder ser curado/validado por responsáveis.

Esses pontos são inferidos a partir dos stakeholders e da descrição técnica.

3 Regras de negócio elicitadas

RN01
O valor da recompensa da questão diminui com o tempo de resposta.

RN02
Após 5 minutos, a questão paga valor mínimo.

RN03
Questões de maior dificuldade pagam mais moedas.

RN04
Cada moeda de questão vale também 1 ponto no ranking.

RN05
O ranking não considera desempenho no Tower Defense.

RN06
Existem duas moedas distintas: moeda de questão e moeda in-game.

RN07
Moeda in-game só vale dentro da partida atual.

RN08
Moeda de questão persiste na carteira do jogador.

RN09
Moedas de questão usadas no Tower Defense não são devolvidas.

RN10
O custo de powerups escala progressivamente com o nível do TD.

RN11
Cosméticos impactam somente a estética, sem efeito sobre gameplay.

RN12
O sistema deve evitar pay-to-win; cosméticos e conveniências devem prevalecer sobre poder direto.

4 Requisitos não funcionais elicitados

RNF01
A aplicação deve ser mobile-first.

RNF02
O frontend deve ser desenvolvido em React com TypeScript.

RNF03
O backend deve ser desenvolvido em Express.js com TypeScript.

RNF04
O banco de dados deve utilizar Supabase/PostgreSQL.

RNF05
A autenticação deve utilizar Supabase Auth.

RNF06
O deploy deve ocorrer em ambiente com Kubernetes.

RNF07
O MVP não depende de WebSocket inicialmente, pois o TD é PvE.

RNF08
A interface deve manter consistência visual entre telas e feedbacks.

5 Lacunas e ambiguidades

- Não define formato exato do cálculo de moedas.
- Não define se questões podem se repetir.

- Não define condições de vitória e derrota.
- Não define HP inicial, quantidade de ondas, balanceamento, custo das torres, árvore de upgrades e regras dos powerups.

- Não define preços dos itens da carteira.

- Não define quais estatísticas serão exibidas.